

BBF 101E 2025-2026 FALL / TERM HOMEWORK

HAZIRLAYAN

ADI SOYADI : HAYRETTİN YAZICI

ÖĞRENCİ NUMARASI : 150240310

TESLİM TARİHİ : 12/12/2025

DERS ADI : INTRODUCTION TO INFORMATION
SYSTEMS

DERS YÜRÜTÜCÜSÜ : NİZAMETTİN AYDIN

INTRODUCTION

1. WHAT IS THE PURPOSE OF THE HOMEWORK

The aim is to write an assembly code using VVM instruction set that reverses my student number.

My student number is 150240310 then I need to obtain 013042051.

2. WHAT IS THE VVM INSTRUCTION SET

Visible Virtual Machine (VVM) is a visual computing environment that allows students to create and observe the execution of their program within a virtual CPU.

The set of VVM that I used is in below:

- **Load Accumulator (5nn) [LDA nn]** The content of RAM address *nn* is copied to the Accumulator Register, replacing the current content of the register. The content of RAM address *nn* remains unchanged. The Program Counter Register is incremented by one.
- **Store Accumulator (3nn) [STO nn] or [STA nn]** The content of the Accumulator Register is copied to RAM address *nn*, replacing the current content of the address. The content of the Accumulator Register remains unchanged. The Program Counter Register is incremented by one.
- **Add (1nn) [ADD nn]** The content of RAM address *nn* is added to the content of the Accumulator Register, replacing the current content of the register. The content of RAM address *nn* remains unchanged. The Program Counter Register is incremented by one.
- **Subtract (2nn) [SUB nn]** The content of RAM address *nn* is subtracted from the content of the Accumulator Register, replacing the current content of the register. The content of RAM address *nn* remains unchanged. The Program Counter Register is incremented by one.
- **Input (901) [IN] or [INP]** A value input by the user is stored in the Accumulator Register, replacing the current content of the register. Note that the two-digit operand does not represent an address in this

instruction, but rather specifies the particulars of the I/O operation (see Output). The operand value can be omitted in the Assembly Language format. The Program Counter Register is incremented by one with this instruction.

- **Output (902) [OUT] or [PRN]** The content of the Accumulator Register is output to the user. The current content of the register remains unchanged. Note that the two-digit operand does not represent an address in this instruction, but rather specifies the particulars of the I/O operation (see Input). The operand value can be omitted in the Assembly Language format. The Program Counter Register is incremented by one with this instruction.
- **Branch if Zero (7nn) [BRZ nn]** This is a conditional branch instruction. If the value in the Accumulator Register is zero, then the current value of the Program Counter Register is replaced by the operand value *nn* (the result is that the next instruction to be executed will be taken from address *nn* rather than from the next sequential address). Otherwise (Accumulator $\neq 0$), the Program Counter Register is incremented by one (thus the next instruction to be executed will be taken from the next sequential address).
- **Branch if Positive or Zero (8nn) [BRP nn]** This is a conditional branch instruction. If the value in the Accumulator Register is nonnegative (i.e., ≥ 0), then the current value of the Program Counter Register is replaced by the operand value *nn* (the result is that the next instruction to be executed will be taken from address *nn* rather than from the next sequential address). Otherwise (Accumulator < 0), the Program Counter Register is incremented by one (thus the next instruction to be executed will be taken from the next sequential address).
- **Branch (6nn) [BR nn] or [BRU nn] or [JMP nn]** This is an unconditional branch instruction. The current value of the Program Counter Register is replaced by the operand value *nn*. The result is that the next instruction to be executed will be taken from address *nn* rather than from the next sequential address. The value of the Program Counter Register is not incremented with this instruction.
- **No Operation (4nn) [NOP] or [NUL]** This instruction does nothing other than increment the Program Counter Register by one. The operand value *nn* is ignored in this instruction and can be omitted in the Assembly Language format. (This instruction is unique to the VVM and is not part of the Little Man Model.)
- **Halt (0nn) [HLT] or [COB]** Program execution is terminated. The operand value *nn* is ignored in this instruction and can be omitted in the Assembly Language format.

- **Embedding Data in Program:** Data values used by a program can be loaded into memory along with the program. In Machine or Assembly Language form simply use the format "*snnn*" where *s* is an optional sign, and *nnn* is the three-digit data value. In Assembly Language, you can specify "*DAT snnn*" for clarity.

THE ALGORITHM

- This algorithm is based on changing the positions of the digits of the student number.
- My student number has 9 digits so I changed 1st-9th, 2nd-8th, 3rd-7th, 4th-6th digits between each other and 5th one (in the middle one) stays same.
- I used temporary memory addresses for replacements.

CODES AND EXPLANATIONS

Set of codes for the program is like that:

00	LDA 50	15	STO 53	30	LDA 53	45	NOP
01	STO 90	16	LDA 90	31	OUT	46	NOP
02	LDA 58	17	STO 58	32	LDA 54	47	NOP
03	STO 50	18	LDA 91	33	OUT	48	NOP
04	LDA 51	19	STO 57	34	LDA 55	49	NOP
05	STO 91	20	LDA 92	35	OUT	50	DAT 001
06	LDA 57	21	STO 56	36	LDA 56	51	DAT 005
07	STO 51	22	LDA 93	37	OUT	52	DAT 000
08	LDA 52	23	STO 55	38	LDA 57	53	DAT 002
09	STO 92	24	LDA 50	39	OUT	54	DAT 004
10	LDA 56	25	OUT	40	LDA 58	55	DAT 000
11	STO 52	26	LDA 51	41	OUT	56	DAT 003
12	LDA 53	27	OUT	42	HLT	57	DAT 001
13	STO 93	28	LDA 52	43	NOP	58	DAT 000
14	LDA 55	29	OUT	44	NOP	--	---

00 LDA 50 → Loading value from address 50 (number 1) into the Accumulator.
01 STO 90 → Storing the value from Accumulator to temporary address 90.
02 LDA 58 → Loading value from address 58 (number 0) into the Accumulator.
03 STO 50 → Storing the value into address 50.

First digit moved to the temporary address and last digit moved to the previous address of first digit.

04 LDA 51	08 LDA 52	12 LDA 53	
05 STO 91	09 STO 92	13 STO 93	→ Same procedure for other pairs
06 LDA 57	10 LDA 56	14 LDA 55	
07 STO 51	11 STO 52	15 STO 53	

16 LDA 90	18 LDA 91	20 LDA 92	22 LDA 93	→ The first four digits moved
17 STO 58	19 STO 57	21 STO 56	23 STO 55	from their temporary
				memory addresses to new ones.

LDA 50 → Loading and outputting the first digit of reversed student number
OUT and it repeated for other addresses (51,52,53,54,55,56,57,58).

42 HLT → It stops the program execution.

43-49 NOP → The program executing no operations, just moving to the next.

50 DAT 001
51 DAT 005
52 DAT 000
53 DAT 002
54 DAT 004 → Definitions of my student number digits to memory addresses 50-58.
55 DAT 000
56 DAT 003
57 DAT 001
58 DAT 000

OUTPUTS OF PROGRAM

The program computes the reversed student number successfully and gives outputs in the correct order.

The screenshot displays a program execution interface with three main panels:

- I/O Log:** A green window showing the execution log. The last line, "[42] End - Normal", is highlighted with a red box.
- Hardware View:** A blue window showing the state of the processor and RAM.
 - PROCESSOR:** Contains four registers: Accum (001), Instr. Reg (HLT), Prog. Ctr (42), and Instruction Cycle (43).
 - RAM:** A table showing memory addresses and their contents.
- Execute:** A control panel with buttons for Run, Step, Pause, Restart, and a speed slider (S to F). It also includes checkboxes for "Show Source Window" and "Tick", and "Help" and "Return" buttons.

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9
0_	550	390	558	350	551	391	557	351	552	392
1_	556	352	553	393	555	353	590	358	591	357
2_	592	356	593	355	550	902	551	902	552	902
3_	553	902	554	902	555	902	556	902	557	902
4_	558	902	000	400	400	400	400	400	400	400
5_	000	001	003	000	004	002	000	005	001	000
6_	000	000	000	000	000	000	000	000	000	000
7_	000	000	000	000	000	000	000	000	000	000
8_	000	000	000	000	000	000	000	000	000	000
9_	001	005	000	002	000	000	000	000	000	000